

NCCL 与 GEMM 跨 Stream 资源竞争的分层归因

脱敏分享版 | 本文讨论多 GPU 训练中 collective 与 GEMM 的资源竞争、代理实验和归因方法，不包含内部拓扑、模型或运行入口。

结论先行

当一个 GEMM 与 NCCL collective 并发后变慢，第一反应往往是“限制 GEMM 使用的 SM”。但一次真实排查说明，必须先回答三个问题：

1. 测量的 GEMM shape 是否真的是 collective 之后的实际 shape？
2. 退化来自 GEMM、通信，还是二者在同一时间窗的资源竞争？
3. 联合配置变快时，收益究竟由通信 CTA 还是 GEMM 配置贡献？

在原训练 trace 中，修正 shape 后约 200 μ s 的 GEMM baseline 属正常范围；跨 stream SendRecv 与 AllGatherV 分别使 GEMM 延长 15.48% 和 28.73%。随后另建四 GPU proxy：经过 121 组粗扫、89 组细扫和候选复验，某 `all_gather(list)` proxy + GEMM 联合配置使 overlap span 降低 12.80%，但同一通信设置下 GEMM 参数的独立贡献只有 7.58%；另一个 `all_to_all_single` proxy 候选看似降低 81.04%，GEMM 独立贡献却仅 1.67%，主因是通信 CTA 配置变化。

前一组退化数字来自原训练 trace；扫参与候选收益来自四 GPU proxy。两类证据回答的问题不同，后者也不是生产 AllGatherV/AllToAllV、跨节点或 full training step 收益。

1. 为什么跨 Stream 不等于真正重叠

CUDA stream 只表达依赖关系和调度机会，不保证两个 kernel 能无损并行。NCCL 与 GEMM 可能竞争：

- SM/CTA residency;
- register file 与 shared memory;
- HBM/L2 带宽;
- copy engine 或互连注入;
- warp scheduler issue slot;
- kernel launch 与同步边界。

```
Stream A:  collective  ██████████
Stream B:           GEMM  ██████████

理想 span:      max(Tcomm, Tgemm)
实际 span:      max(...) + contention + sync overhead
```

因此分析目标不只是两个 kernel 各自耗时，而是它们的联合时间窗，以及每个配置对这段 span 的独立贡献。

2. 第一步：先纠正真实 shape

多 GPU MoE 路径中，FC1 的输入通常会在 dispatch、AllGather 或 AllToAll 后改变 token 行数。如果 benchmark 使用了 collective 前的 shape，就可能得到完全错误的基线，再把正常的后 collective GEMM 误判成 kernel 退化。

正确做法是沿数据流记录：

```
local tokens
-> routing / padding
-> collective
-> post-collective rows
-> FC1 GEMM
```

每一处都核对 tensor shape、dtype、layout 与有效行数。修正到真实 post-collective shape 后，约 200 μ s 的单 GEMM baseline 与硬件能力、数据量和成熟实现的量级一致，排除了“GEMM 本体异常”。

3. 第二步：建立四类基线

只比较“通信开/关”仍不够。至少需要：

组别	Stream A	Stream B	回答的问题
G	空	GEMM	GEMM 独立基线
C	Collective	空	通信独立基线
CG-same	Collective	GEMM	同 stream 串行上界
CG-cross	Collective	GEMM	跨 stream 实际 overlap

关键指标包括：

```
contention ratio = T_cross_gemm / T_gemm_alone - 1
overlap span      = end(max) - start(min)
overlap efficiency= (Tcomm + Tgemm - Tspan) / min(Tcomm, Tgemm)
```

原训练 trace 的对照中，跨 stream SendRecv 与 AllGatherV 分别使 GEMM 时间增加 15.48% 和 28.73%，说明该受测时间窗内存在通信—计算资源竞争。这两个比例不来自后续四 GPU proxy 扫参。

4. 第三步：把搜索拆成通信参数与 GEMM 参数

4.1 为什么联合扫参容易误判

假设同时修改：

- NCCL 每 channel 的 CTA/request 设置；
- GEMM 可用 SM 数或 tile 配置。

如果联合 span 大幅下降，不能直接说“限制 GEMM SM 成功”。通信参数本身可能减少了排队、改善并行度，甚至是全部收益来源。

4.2 分阶段搜索

随后在四 GPU proxy 中采用以下搜索流程。通信侧分别使用 `all_gather(list)` 和 `all_to_all_single` 构造代理负载；它们用于复现相近的通信—计算竞争，不等同于生产 trace 中的 AllGatherV/AllToAllV 实现。

采用的搜索流程是：

1. 121 组 coarse sweep，定位通信/GEMM 的可行区间；
2. 89 组 fine sweep，在候选附近细化；

3. 对每个候选固定通信参数，只改变 GEMM 参数，测独立贡献；
4. 再固定 GEMM 参数，只改变通信参数；
5. 多轮复验联合 span，而不是只看单次最优。

5. 两个代表性结果

5.1 `all_gather(list)` proxy 候选

四 GPU `all_gather(list)` proxy 中，某通信 CTA 与 GEMM SM 配置的组合使 joint span 下降 12.80%。在完全相同的通信设置下，仅替换 GEMM 配置仍有 7.58% 的独立贡献。因此可以说：

- 联合配置有效；
- GEMM 限制贡献了部分收益；
- 其余收益来自通信配置及二者交互。

5.2 `all_to_all_single` proxy 候选

四 GPU `all_to_all_single` proxy 中，另一候选的 joint span 下降 81.04%，数字非常醒目；但固定通信配置后，GEMM 的独立贡献只有 1.67%。因此主要收益来自通信 CTA，而非 GEMM SM 配置。

如果只保存“81.04%”而不做独立贡献实验，结论会完全错位。

6. Summed work 与 Critical span 不能混用

Profiler 常同时提供：

- 各 kernel duration 的求和；
- 时间线上实际重叠后的 wall span；
- 单个 stream 的累计执行时间。

当两个 kernel 并行时，summed duration 可以大于 wall time。优化 overlap 的最终指标应是 critical span 或完整 step，而不是所有 kernel duration 的简单和。

```
Kernel A: |----- 10 ms -----|
Kernel B:      |----- 8 ms -----|

summed work = 18 ms
wall span   < 18 ms
```

同样，Nsys 带来的 projected duration 只适合结构分析，不应与无 profiler 的 CUDA Event 结果拼接。

7. 为什么这里只能叫 Proxy

四 GPU 单节点 proxy 能回答：

- 当前 shape 下是否存在通信—计算竞争；
- 哪些 CTA/SM 区间值得进入真实实验；
- 收益主要来自通信还是 GEMM。

其中 `all_gather(list)` / `all_to_all_single` 只是代理 API，并不复刻生产 trace 中 collective 的全部协议、kernel 与拓扑行为。它不能回答：

- 跨节点网络与真实拓扑是否相同；
- 完整 MoE 层的其他通信会不会抵消收益；

- 多层 pipeline 中是否改变关键路径；
- full training step 是否改善；
- 长跑稳定性和不同 batch 是否覆盖。

因此结果应写成“候选配置/代理 span”，而不是“训练提速”。

8. 推荐的归因表

四 GPU proxy 配置	通信参数	GEMM 参数	Comm-only	GEMM-only	Joint span	独立贡献
Baseline	C0	G0	归一化	归一化	1.000	—
<code>all_gather(list)</code> Candidate A	C1	G1	分别测	分别测	0.872	GEMM 约 7.58%
<code>all_to_all_single</code> Candidate B	C2	G2	分别测	分别测	0.190	GEMM 约 1.67%

表中最重要的一列不是 joint span，而是“独立贡献”。它迫使实验者为因果表述提供 matched control。

9. 可复用检查表

- ☐ benchmark shape 是否来自 collective 后的真实 tensor？
- ☐ 是否分别测 GEMM-only、comm-only、same-stream 与 cross-stream？
- ☐ 是否区分原训练 trace 与代理 collective API 的证据？
- ☐ 指标是 summed work、单 kernel latency，还是 critical span？
- ☐ 通信 CTA 与 GEMM SM 参数是否做了独立控制？
- ☐ 联合最优是否经过多轮复验？
- ☐ 是否区分单节点 proxy 与跨节点 full step？
- ☐ 是否避免把通信参数收益归因给 GEMM？

10. 总结

通信—计算重叠优化最危险的不是“没找到更快配置”，而是找到一个快很多的联合配置后给出错误归因。正确流程是先用原训练 trace 核对真实 shape 与退化，再在明确标注 API 和资源范围的 proxy 中建立单独与联合基线，最后用 matched control 拆解通信和 GEMM 的独立贡献。`all_gather(list)`/`all_to_all_single` proxy 只能筛选候选；只有候选进入真实 collective、真实拓扑和完整 step 后，才有资格成为训练收益。